



GRAPH:

A graph G consists of two things:

- (1) A set V of elements called *nodes* (or *points* or *vertices*)
- (2) A set E of *edges* such that each edge e in E is identified with a unique (unordered) pair $[u, v]$ of nodes in V , denoted by $e = [u, v]$

If $e=[u, v]$ then

e stands for an edge,

u and v are end points of e

u and v are vertex

u and v are adjacent nodes or neighbors

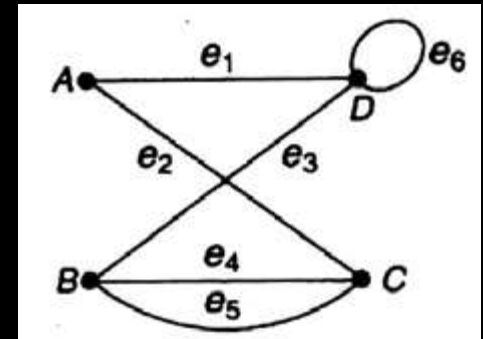
A graph G is represented by: $G = (V, E)$.

Where,

$V=(A, B, C, D)$ and

$E=(e_1, e_2, e_3, e_4, e_5, e_6)$

$e_1=[A, D]$, $e_2=[A, C]$, $e_3=[B, D]$, $e_4=[B, C]$, $e_5=[B, C]$, $e_6=[D, D]$





CSE 203: Data Structure

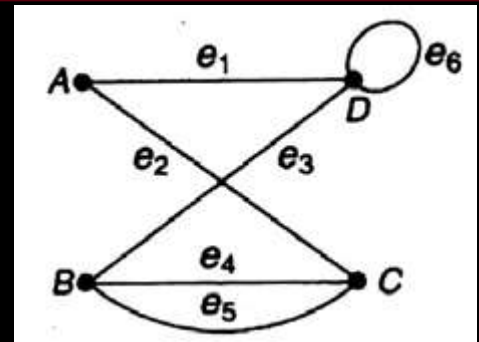
Data
Structure

GRAPH:

Degree of a node u , written as $\deg(u)$, is the number of edges that meet at u .

Here, $\deg(A)=2$, $\deg(B)=3$

Again $\deg(u)=0$ means that u is isolated and has no edge



A path P of length n from a node u to a node v is defined as a sequence of $n + 1$ nodes.

$$P = (v_0, v_1, v_2, \dots, v_n)$$

The path P is said to be *closed* if $v_0 = v_n$.

A *cycle* is a closed simple path with length 3 or more.

A cycle of length k is called a *k-cycle*.

A graph G is said to be *connected* if there is a path between any two of its nodes.

A graph G is said to be *complete* if every node u in G is adjacent to every other node v in G .

A connected graph T without any cycles is called a *tree graph* or *free tree* or, simply, a *tree*.

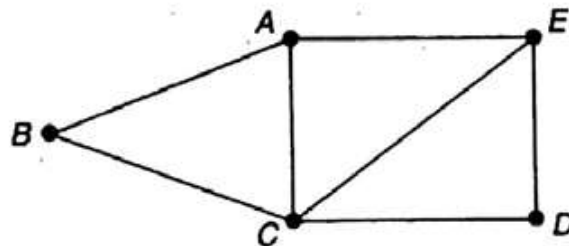
A graph G is said to be *labeled* if its edges are assigned data.



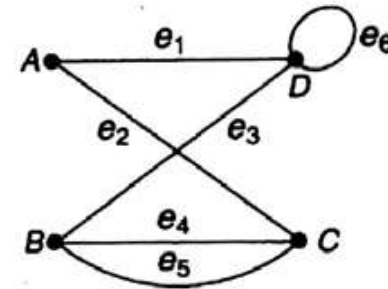
GRAPH:

- (1) *Multiple edges*. Distinct edges e and e' are called *multiple edges* if they connect the same endpoints, that is, if $e = [u, v]$ and $e' = [u, v]$.
- (2) *Loops*. An edge e is called a *loop* if it has identical endpoints, that is, if $e = [u, u]$.

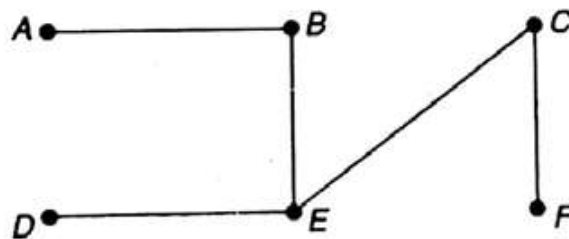
Such a generalization M is called a *multigraph*.



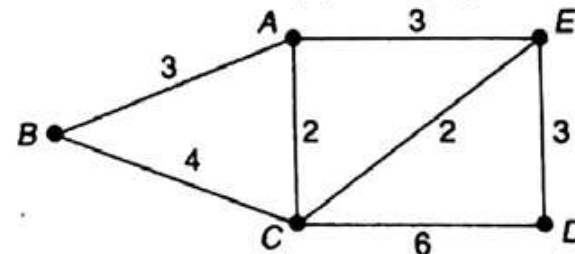
(a) Graph



(b) Multigraph



(c) Tree



(d) Weighted graph



GRAPH:

Directed Graphs

A *directed graph* G , also called a *digraph* or *graph*, is the same as a multigraph except that each edge e in G is assigned a direction, or in other words, each edge e is identified with an ordered pair (u, v) of nodes in G rather than an unordered pair $[u, v]$.

$e = (u, v)$

e is also called an *arc*

- (1) e begins at u and ends at v .
- (2) u is the *origin* or *initial point* of e , and v is the *destination* or *terminal point* of e .
- (3) u is a *predecessor* of v , and v is a *successor* or *neighbor* of u .
- (4) u is *adjacent to* v , and v is *adjacent to* u .

outdegree of a node u $\rightarrow$ outdeg(u)

indegree of u $\rightarrow$ indeg(u)

A node v is said to be *reachable* from a node u if there is a (directed) path from u to v .

(A directed graph G is said to be *connected*, or *strongly connected*, if for each pair u, v of nodes in G there is a path from u to v and there is also a path from v to u . On the other hand, G is said to be *unilaterally connected* if for any pair u, v of nodes in G there is a path from u to v or a path from v to u .)



CSE 203: Data Structure

Data
Structure

GRAPH:

Sequential representation of a Graph

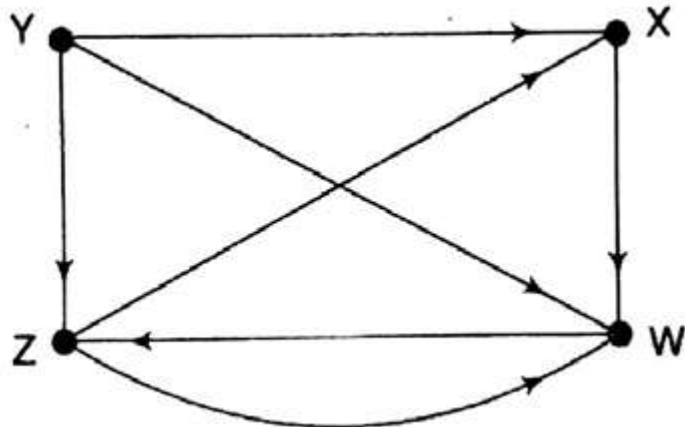
- Adjacent matrix
- Path matrix
- Weighted matrix

Let $v_1, v_2, \dots, v_m$ vertices in a graph G

Size of adjacent matrix is $m \times m$

Let the adjacent matrix is $A=(a_{i,j})$

$$a_{ij} = \begin{cases} 1 & \text{if } v_i \text{ is adjacent to } v_j, \text{ that is, if there is an edge } (v_i, v_j) \\ 0 & \text{otherwise} \end{cases}$$



$$A = \begin{matrix} & \begin{matrix} X & Y & Z & W \end{matrix} \\ \begin{matrix} X \\ Y \\ Z \\ W \end{matrix} & \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \end{matrix}$$



CSE 203: Data Structure

Data
Structure

GRAPH:

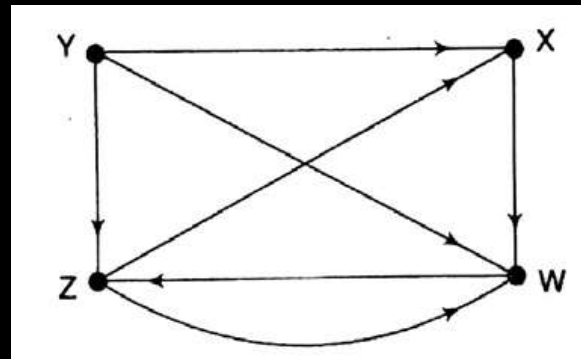
Sequential representation of a Graph

- Adjacent matrix
- Path matrix
- Weighted matrix

Let $v_1, v_2, \dots, v_m$ vertices in a graph G

Size of adjacent matrix is $m \times m$

Let the adjacent matrix is $A=(a_{i,j})$



$$A = \begin{matrix} & \begin{matrix} X & Y & Z & W \end{matrix} \\ \begin{matrix} X \\ Y \\ Z \\ W \end{matrix} & \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \end{matrix}$$

Consider the powers $A, A^2, A^3, \dots$ of the adjacency matrix A of a graph G . Let

$a_K(i, j)$ = the ij entry in the matrix A^K

$a_1(i, j) = a_{ij}$ gives the number of paths of length 1 from node v_i to node v_j .

$a_2(i, j)$ gives the number of paths of length 2 from v_i to v_j .



GRAPH:

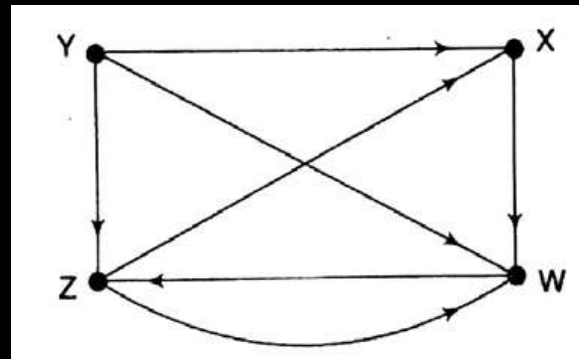
Sequential representation of a Graph

- Adjacent matrix
- Path matrix
- Weighted matrix

Let $v_1, v_2, \dots, v_m$ vertices in a graph G

Size of adjacent matrix is $m \times m$

Let the adjacent matrix is $A=(a_{i,j})$



$$A = \begin{matrix} & \begin{matrix} X & Y & Z & W \end{matrix} \\ \begin{matrix} X \\ Y \\ Z \\ W \end{matrix} & \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \end{matrix}$$

$$A^2 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 2 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix}$$

$$A^3 = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 1 & 0 & 2 & 2 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

$$A^4 = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 2 & 0 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

$$B_r = A + A^2 + A^3 + \dots + A^r$$

B_r gives the number of paths of length r or less from node v_i to v_j .



CSE 203: Data Structure

Data
Structure

GRAPH:

Sequential representation of a Graph

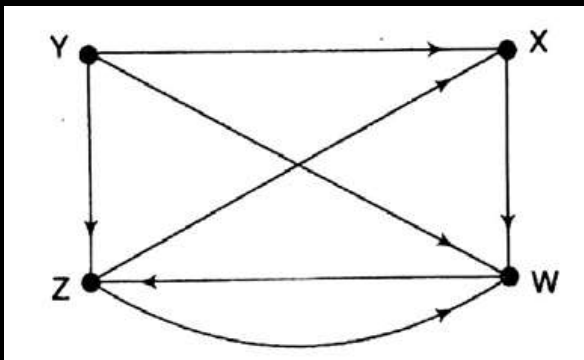
- Adjacent matrix
- Path matrix
- Weighted matrix

Let $v_1, v_2, \dots, v_m$ vertices in a graph G

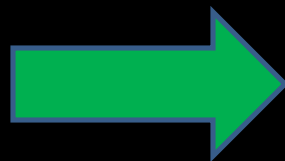
Size of adjacent matrix is $m \times m$

Let the path matrix is $P=(p_{ij})$

$$P_{ij} = \begin{cases} 1 & \text{if there is a path from } v_i \text{ to } v_j \\ 0 & \text{otherwise} \end{cases}$$



$$B_4 = \begin{pmatrix} 1 & 0 & 2 & 3 \\ 5 & 0 & 6 & 8 \\ 3 & 0 & 3 & 5 \\ 2 & 0 & 3 & 3 \end{pmatrix}$$



$$P = \begin{matrix} & \begin{matrix} X & Y & Z & W \end{matrix} \\ \begin{matrix} X \\ Y \\ Z \\ W \end{matrix} & \begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix} \end{matrix}$$

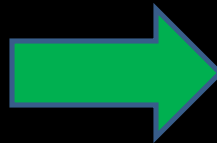
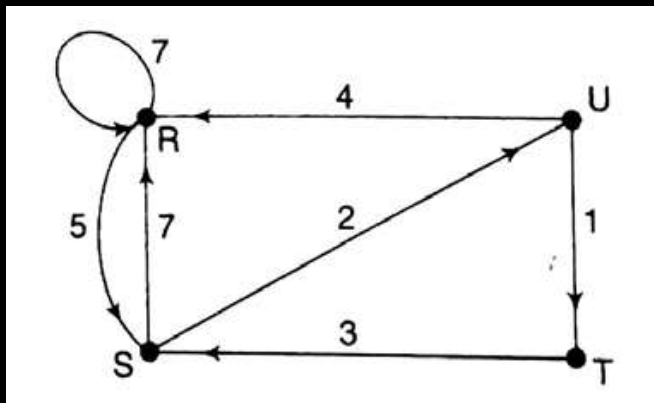


GRAPH:

Sequential representation of a Graph

- Adjacent matrix
- Path matrix
- Weighted matrix

$$w_{ij} = \begin{cases} w(e) & \text{if there is an edge } e \text{ from } v_i \text{ to } v_j \\ 0 & \text{if there is no edge from } v_i \text{ to } v_j \end{cases}$$



$$W = \begin{pmatrix} 7 & 5 & 0 & 0 \\ 7 & 0 & 0 & 2 \\ 0 & 3 & 0 & 0 \\ 4 & 0 & 1 & 0 \end{pmatrix}$$

Finding shortest path:

- Repeat for I, J = 1, 2, ..., M: [Initializes Q.]
 W[I, J] = 0, then: Set Q[I, J] := INFINITY;
 Else: Set Q[I, J] := W[I, J].
[End of loop.]

$$Q_0 = \begin{pmatrix} 7 & 5 & \infty & \infty \\ 7 & \infty & \infty & 2 \\ \infty & 3 & \infty & \infty \\ 4 & \infty & 1 & \infty \end{pmatrix}$$



CSE 203: Data Structure

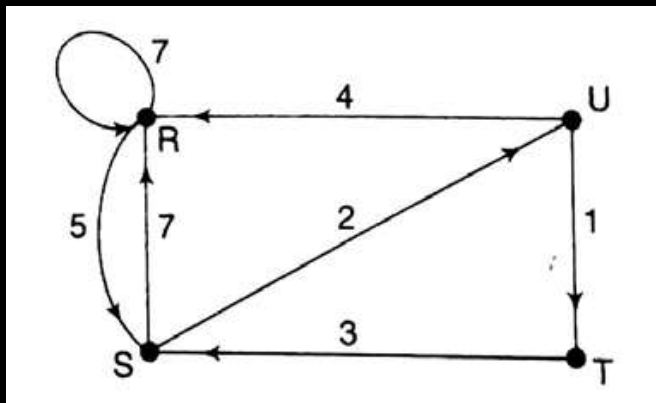
Data
Structure

GRAPH:

Sequential representation of a Graph

- Adjacent matrix
- Path matrix
- Weighted matrix

$$w_{ij} = \begin{cases} w(e) & \text{if there is an edge } e \text{ from } v_i \text{ to } v_j \\ 0 & \text{if there is no edge from } v_i \text{ to } v_j \end{cases}$$



Finding shortest path:

$$Q_0 = \begin{pmatrix} 7 & 5 & \infty & \infty \\ 7 & \infty & \infty & 2 \\ \infty & 3 & \infty & \infty \\ 4 & \infty & 1 & \infty \end{pmatrix}$$

Do for all i and j:

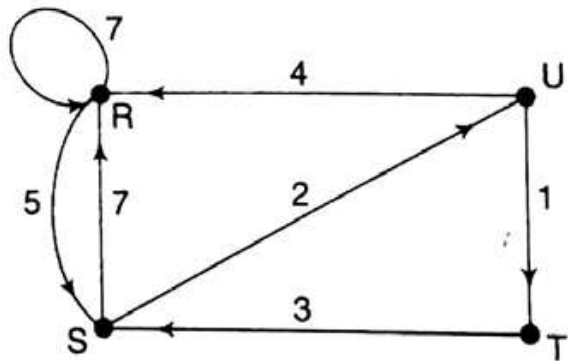
$$Q_k[i, j] = \min(Q_{k-1}[i, j], Q_{k-1}[i, k] + Q_{k-1}[k, j])$$



GRAPH:

Sequential representation of a Graph

- Adjacent matrix
- Path matrix
- Weighted matrix



Finding shortest path:

$$Q_0 = \begin{pmatrix} 7 & 5 & \infty & \infty \\ 7 & \infty & \infty & 2 \\ \infty & 3 & \infty & \infty \\ 4 & \infty & 1 & \infty \end{pmatrix}$$

$$w_{ij} = \begin{cases} w(e) & \text{if there is an edge } e \text{ from } v_i \text{ to } v_j \\ 0 & \text{if there is no edge from } v_i \text{ to } v_j \end{cases}$$

$$Q_k[i, j] = \text{MIN}(Q_{k-1}[i, j], Q_{k-1}[i, k] + Q_{k-1}[k, j])$$

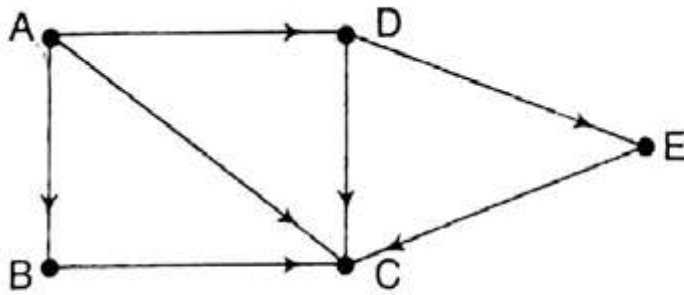
2. Repeat Steps 3 and 4 for $K = 1, 2, \dots, M$: [Updates Q.]
3. Repeat Step 4 for $I = 1, 2, \dots, M$:
4. Repeat for $J = 1, 2, \dots, M$:
Set $Q[I, J] := \text{MIN}(Q[I, J], Q[I, K] + Q[K, J])$.
[End of loop.]
[End of Step 3 loop.]
[End of Step 2 loop.]
5. Exit.



GRAPH:

LINKED REPRESENTATION OF A GRAPH

Using adjacency list,



(a) Graph G

Node	Adjacency List
A	B, C, D
B	C
C	D, E
D	E
E	

(b) Adjacency list of G